

Item 3 & Item 5

Item 3 — 会话卡片缓存 50 条、增量更新与冷加载

apps/web/src/hooks/use-web-socket.ts — `invalidateRepositoryRealtimeQueries()` 移除

`workbenchQueryKeys.conversationMessagesRoot()` 全量 `invalidateQueries`，改为按 `repositoryId` 定向 `refetchQueries()`，`predicate` 匹配 `key[0]=== 'workbench' && key[1]=== 'conversation-messages' && key[2]=== repositoryId`，`type: 'active'` 仅刷新活跃查询

apps/web/src/pages/DesktopWorkbench.tsx — `RepositoryConversation` 新增

`coldLoadSentinelRef` + `coldLoadGuardRef` (防重复触发)，`IntersectionObserver` 监听哨兵 `div` 可见性，`rootMargin: '200px'` 提前触发，`isIntersecting` 时自动调用 `onLoadOlderMessages()`，`loadingOlderMessages` 结束后重置 `guard`

1. `coldLoadSentinelRef` 哨兵 `div` 置于消息列表底部「加载更多消息」按钮上方，`className="h-pw-full" aria-hidden`

apps/web/src/pages/DesktopWorkbench.tsx — `MAX_CONVERSATION_MESSAGES = 100`

`selectedMessages` `useMemo` 中 `.slice(0, 100)`；`filteredMessages` 渲染中 `.slice(0, MAX_CONVERSATION_MESSAGES)`；`handleLoadOlderConversationMessages` 中 `merged.slice(merged.length - 100)` 裁剪；超出容量时底部提示「已显示最近 100 条消息，向上滚动加载更多」

apps/web/src/pages/DesktopWorkbench.tsx — `handleLoadOlderConversationMessages` 调用

`workbenchService.getConversationMessages(repositoryId, { cursor: conversationNextCursor, take: CONVERSATION_MESSAGE_PAGE_SIZE })`，追加到 `olderConversationMessages` 并裁剪至 100 条，更新 `conversationNextCursor`

Item 5 — 解决 Agent 输入框延迟瓶颈

apps/web/src/pages/DesktopWorkbench.tsx — `AgentChatInputField` (新组件) 自管理 `input state (useState)`，`handleSend` 用 `useCallback` 包装 (`deps: [input, hasSessionPrompt, hasApiKey, onSend]`)，Enter 键 / Run 按钮 → `setInput('') + onSend(trimmed)`，Stop 按钮 → `onStop()`

1. `props: { onSend: (prompt: string) => void; onStop: () => void; isRunning: boolean; hasApiKey: boolean; hasSessionPrompt: boolean }`
2. Run 按钮 `disabled={!input.trim() && !hasSessionPrompt} || !hasApiKey`
3. 无 API Key 时显示 `AlertTriangle` 警告提示

apps/web/src/pages/DesktopWorkbench.tsx — `AgentRunView` 移除 `const [chatInput,`

`setChatInput] = useState('')`、`setChatInput('')` 调用、`Input value={chatInput} onChange={(e) => setChatInput(e.target.value)}`

apps/web/src/pages/DesktopWorkbench.tsx — `handleSendChat(overridePrompt?)` `const`

`userPrompt = (overridePrompt ?? chatInput).trim()` 改为 `const userPrompt = overridePrompt?.trim() ?? ''`，不再依赖父组件 `chatInput state`，不调用 `setChatInput`

apps/web/src/pages/DesktopWorkbench.tsx — 输入区域 JSX 原 70+ 行内联 `Input` + `Button` 替换为 `<AgentChatInputField` `onSend`={ (prompt) => handleSendChat(prompt)} `onStop`={ () => stopSessionOnSession(activeSession)} `isRunning`={activeSession.status === 'running' || activeSession.status === 'waiting_permission'} `hasApiKey`={Boolean(activeApiKey)} `hasSessionPrompt`={Boolean(activeSession.prompt)} `/>`

apps/web/src/pages/DesktopWorkbench.tsx — `GitTreePanel` `onAskAgent` `setChatInput(prompt)` 改为 `handleSendChat(prompt)` , 直接从 Git 树发送 prompt 到 Agent

apps/web/src/pages/DesktopWorkbench.tsx — `import` 新增 `useCallback` 到 React `import`

Item 3 效果

- `Socket` `event:new` 到来时不再全量失效所有会话消息缓存, 仅定向 `refresh` 当前活跃仓库
- 用户滚动到消息列表底部 → `IntersectionObserver` 自动触发冷加载, `rootMargin` `200px` 提前预加载
- `olderConversationMessages` 追加后自动裁剪, 前端内存中最多保留 100 条
- 超出容量时显示提示, 用户可继续向下滚动触发冷加载

Item 5 效果

- `keystroke` `re-render` 开销降为 0: 输入框每次按键仅触发 `AgentChatInputField` (~60 行组件) `re-render`
- 父组件 `AgentRunView` (命令行、日志、文件树、Git 可视化等大量 DOM) 完全不受影响
- 行为字节级一致: `Enter`/按钮发送、`Stop` 停止、`API Key` 警告、重试逻辑均保持
- `onAskAgent` 从预填输入框改为直接发送 `prompt`, 交互更直接